

Garbage Collection in .NET Framework

In this article, I will discuss **Garbage Collection in .NET Framework** with Examples. Please read our previous article discussing [Destructor in C#](#) with Examples. At the end of this article, you will understand what a garbage collector is in the .NET and how it works. As part of this article, we will discuss the following pointers in detail.

1. **What is Garbage Collection in C#.NET?**
2. **What are the different Generations of Garbage Collections?**
3. **How do you use the .NET Memory Profiler to check different garbage collection generations?**
4. **How does using a destructor in a class end up in a double garbage collection loop?**
5. **How can we solve the double loop problems using Finalize Dispose Patterns?**

What is Garbage Collection in .NET Framework?

When a dot net application runs, lots of objects are created. At a given point in time, it is possible that the application does not use some of those objects. The Garbage Collector in the .NET Framework is nothing but a Small Routine, or you can say it's a Background Process Thread that runs periodically and tries to identify what objects are not being used currently by the application and de-allocates the memory of those objects.

So, Garbage Collector is nothing but a feature provided by CLR that helps us clean or destroy unused managed objects. Cleaning or destroying those unused managed objects basically reclaims the memory.

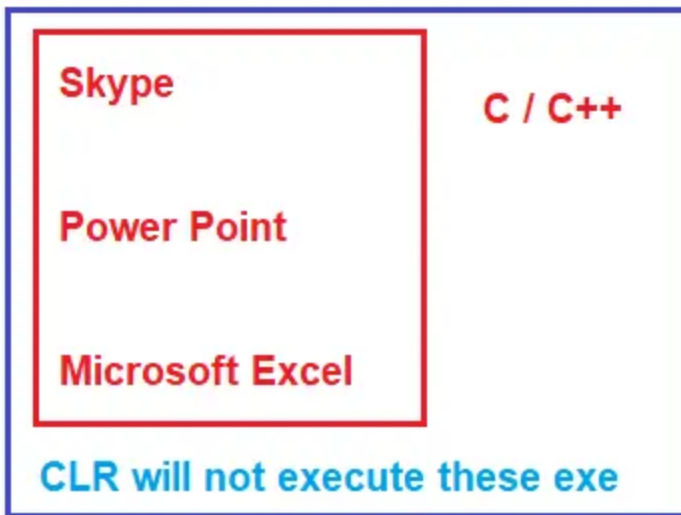
Garbage Collection (GC) in the .NET Framework is an automatic memory management system that helps manage the allocation and release of memory in your applications. In .NET, when we create an object using the new keyword, it automatically allocates memory on the managed heap. You don't need to allocate or deallocate memory explicitly as you might have to in languages like C or C++.

Note: The Garbage Collector will destroy only the unused managed objects. It does not clean unmanaged objects.

Managed and Unmanaged Objects in .NET Framework:

Let us understand Managed and Unmanaged objects. Whenever we create any EXE (i.e., Console Application, Windows Application, etc.) or Web Application (i.e., ASP.NET MVC, Web API, ASP.NET, Class Library, etc.) in .NET Framework using Visual Studio and using any .NET supported programming language such as C#, VB, F#, etc. These applications are run completely under the control of CLR (Common Language Runtime). That means if your applications have unused objects, then CLR will clean those objects using Garbage Collector.

Let's say you have also used other third-party EXE in your .NET application, like Skype, PowerPoint, Microsoft Excel, etc. These "EXEs" are not made in dot net. They are made using some other programming languages such as C, C++, Java, etc.



When you use these “EXEs” in your application, then these are not run by CLR. Even though you run these “EXE” in dot net applications, they will run under their own environment. For example, if one EXE is developed using C or C++, then that EXE will run under the C or C++ Runtime Environment. In the same line, if the EXE is created using VB6, then it will run under the VB6 runtime environment.

The codes that run under the complete control of CLR are called Managed Code in the .NET Framework. CLR will provide all the facilities and features of .NET to the managed code execution like Language Interoperability, Automatic Memory Management, Exception Handling Mechanism, Code Access Security, etc.

On the other hand, Skype, PowerPoint, and Microsoft Excel do not require dot net runtime. They run under their own environment. So, in short, the code (EXE, Web App) that is not run under the control of CLR is called unmanaged code. CLR will not provide any facilities and features of .NET to the unmanaged code in C# execution like Language Interoperability, Automatic memory management, Exception handling mechanism, code access security, etc.

Managed Objects:

Managed objects are allocated on the managed heap and controlled by the .NET Garbage Collector (GC). These objects are typically instances of classes and structures defined in .NET. The GC automatically manages the memory for managed objects. It allocates and releases memory for these objects and handles memory optimizations like compacting. Examples of managed objects are any instance of a class or structure created using the new keyword in C# or VB.NET and objects created in .NET languages like arrays, strings, etc.

Unmanaged Objects:

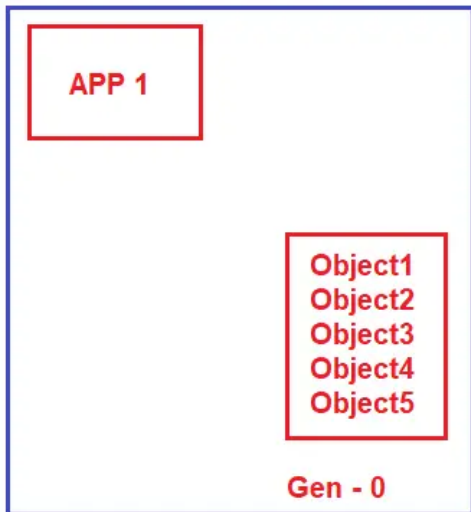
Unmanaged objects are objects whose memory is not managed by the .NET GC. These are typically objects allocated using native code, like calls to Windows API or using languages such as C or C++. The developer is responsible for allocating and freeing the memory for unmanaged objects. This is usually done using APIs like malloc and free in C or new and delete in C++. Examples of unmanaged objects are file handles, database connections, COM objects, or any other resources that are not managed by the .NET runtime.

Garbage Collection Generations in .NET Framework:

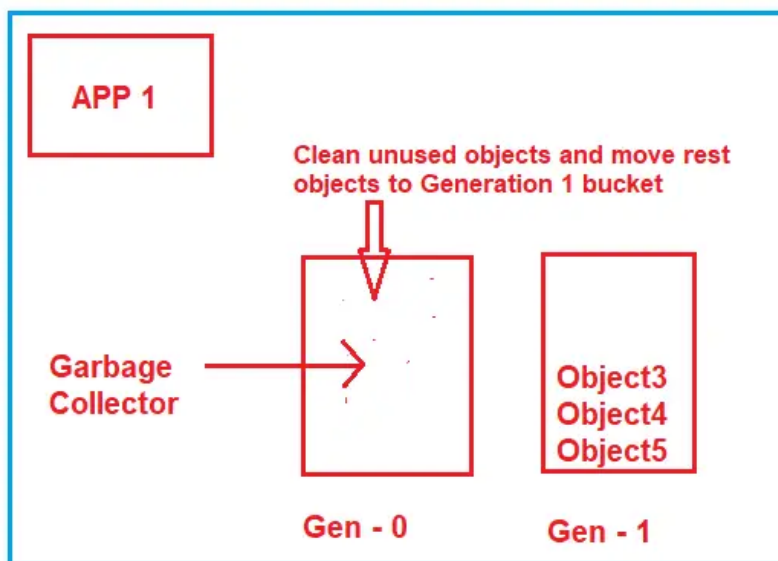
Let us understand what garbage collector generations are and how they affect garbage collection performance. There are three generations. They are Generation 0, Generation 1, and Generation 2.

Understanding Generation 0, 1, and 2:

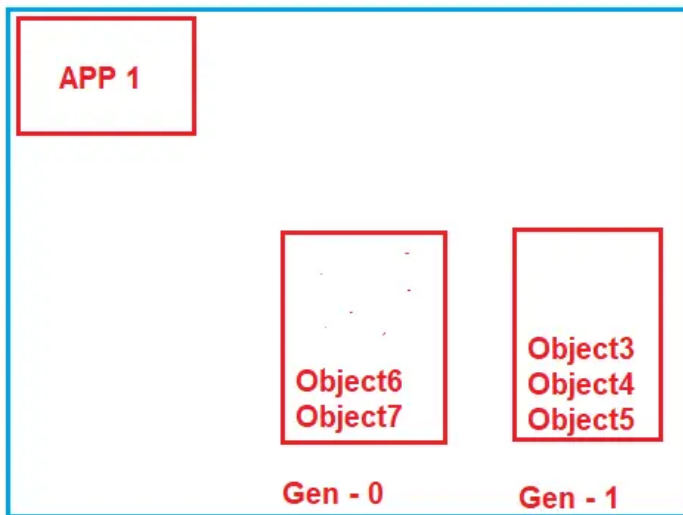
Let's say you have a simple application called App1. As soon as the application starts, it creates 5 managed objects. Whenever any new objects (fresh objects) are created, they are moved into a bucket called Generation 0. For a better understanding, please have a look at the following image.



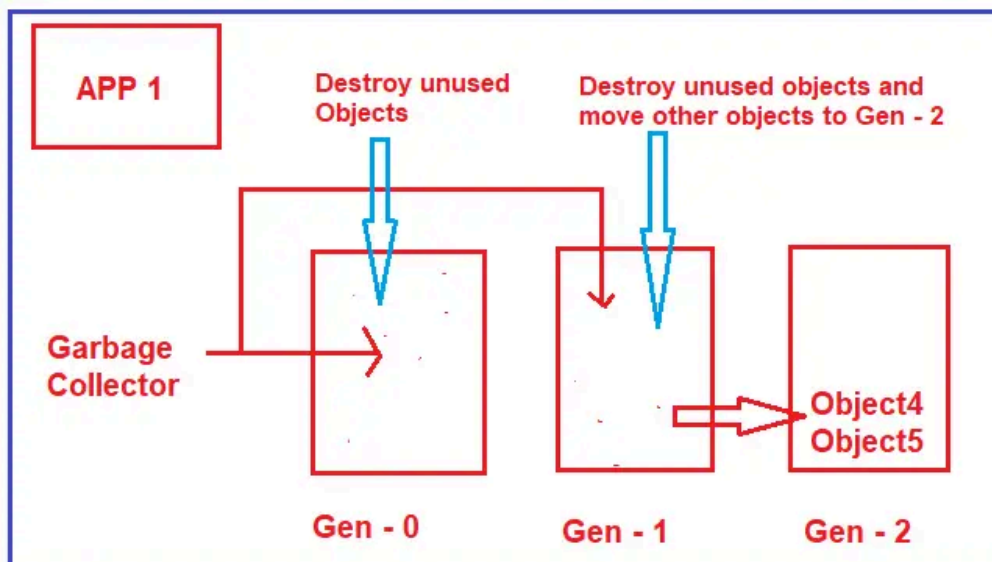
We know our hero, Mr. Garbage Collector, runs continuously as a background process thread to check whether there are any unused managed objects so that it reclaims the memory by cleaning those objects. Now, let's say the application does not need two objects (Object1 and Object2). So, the Garbage Collector will destroy these two objects (Object1 and Object2) and reclaim the memory from the Generation 0 bucket. But the application still needs the remaining three objects (Object3, Object4, and Object5). So, the Garbage Collector will not clean those three objects. The Garbage Collector will move those three managed objects (Object3, Object4, and Object5) to the Generation 1 bucket, as shown in the image below.



Let's say your application creates two more fresh objects (Object6 and Object7). As fresh objects, they should be created in the Generation 0 bucket, as shown in the image below.



Now, again, the Garbage Collector runs, and it comes to the Generation 0 bucket and checks which objects are used. Let's say both objects (Object6 and Object7) are unused by the application, so it will remove both objects and reclaim the memory. Now, it goes to the Generation 1 bucket and checks which objects are unused. Let's say the application still needs Object4 and Object5 while object3 is not needed. So, what Garbage Collector will do is destroy Object3, reclaim the memory, and move Object4 and Object5 to the Generation 2 bucket, as shown in the image below.



What are Generations?

Generations are nothing but will define how long the objects stay in the memory. Now, the question that should come to your mind is why do we need Generations? Why do we have three different kinds of generations?

Why do we need Generations?

Normally, when we are working with big applications, they can create thousands of objects. So, for each of these objects, if the garbage collector goes and checks if they are needed, it's a really painful or bulky process. By creating such generations, what does it mean if an object in Generation 2 buckets means the

Garbage Collector will do fewer visits to this bucket? The reason is that if an object moves to Generation 2, it will stay in the memory longer. There's no point going and checking them again and again.

So, in simple words, we can say that Generations 0, 1, and 2 will help increase the Garbage Collector's performance. The more the objects in Gen 0, the better the performance and the more the memory will be utilized in an optimal manner.

Note: To better clarify the generations, we will use a tool called **.NET Memory Profiler**. Now, I will show you how to download, install, and use .NET Memory Profiler with C# Console Application to check and see how the objects are created in the different generations of Garbage Collector.

Garbage Collection Generations in .NET Framework

Garbage Collection (GC) in the .NET Framework uses a generational model to manage memory more efficiently. This model organizes objects into three generations based on their lifetimes and survival rate through GC cycles. Here's a detailed overview of each generation:

Generation 0

1. **Short-Lived Objects:** Generation 0 (Gen 0) contains newly created objects that are short-lived. These are typically temporary objects.
2. **Frequent Collection:** Gen 0 is collected more frequently than the other generations. Most objects are reclaimed for garbage collection in this generation since they are short-lived.
3. **Efficiency:** The collection process is generally fast because it involves only a small portion of the heap.
4. **Example:** Temporary variables, small objects, and objects that are used briefly in methods.

Generation 1

- **Buffer Generation:** Generation 1 (Gen 1) serves as a buffer between short-lived objects in Gen 0 and long-lived objects in Gen 2.
- **Promotion:** Objects that survive a Gen 0 GC are promoted to Gen 1. These are typically objects that have a longer lifetime than those in Gen 0 but are not permanent.
- **Intermediate Lifetime:** Gen 1 is collected less frequently than Gen 0. The objects here have an intermediate lifetime.
- **Example:** Objects that survive several GC cycles but are not used throughout the application's lifetime.

Generation 2

- **Long-Lived Objects:** Generation 2 (Gen 2) contains long-lived objects. These are objects that have survived multiple rounds of GC in the previous generations.
- **Least Frequently Collected:** Gen 2 is collected less frequently than Gen 0 and Gen 1. The GC process here can be more time-consuming because it involves a larger portion of the heap.
- **Examples:** Static objects, objects tied to the application's life, and large objects that require significant memory resources.

What is the .NET Memory Profiler?

.NET Memory Profiler is a powerful tool for finding memory leaks and optimizing memory usage in programs written in C#, VB.NET, or any other .NET Language. With the help of the profiling guides, the automatic memory analyzer, and specialized trackers, you can make sure that your program has no memory or resource leaks and that the memory usage is as optimal as possible.

How to Download .NET Memory Profiler?

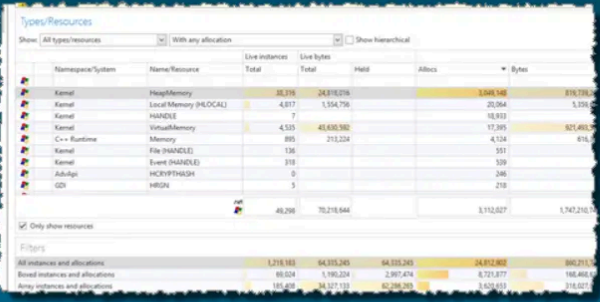
To download the .NET Memory Profiler, please visit the following link.

<https://memprofiler.com/>

Once you click on the above link, it will open the following webpage. From the page below, click on the Download Free Trial button as shown in the image below.

.NET Memory Profiler

[Home](#)[Download](#)[Buy](#)[Support](#)[Forum](#)[About](#)



NameSpace/System	Name/Resource	Total	Live Bytes	Peak	Allocs	Bytes
Kernel	PrivateMemory	38,376	24,818,216		8,945,548	319,756,344
Kernel	Local Memory (LOCAL)	4,817	1,554,736		20,864	1,128,456
Kernel	HANDLE	7		16,833		
Kernel	VirtualMemory	4,533	41,636,582		17,305	521,493,592
C++ Runtime	Memory	895	213,224		4,124	832,512
Kernel	File (HANDLE)	138		393		
Kernel	Event (HANDLE)	318		539		
Advapi	hCRYPTFHA204	0		246		
GDI	hGDIP	3		219		
		48,290	75,218,544		3,112,627	1,742,216,736

Reduce resource usage

Click Here to Download

Download Free Trial or get Pricing information

Once you click the Download Free Trial button, another page asking you to enter your email address will open. You can enter the email address or click on the Download button to download the .NET Memory Profiler, as shown in the image below.

Download .NET Memory Profiler 5.7 Installer

Email address (optional):

Enter your email address to be sent occasional emails about upgrades, special offers and other relevant information.

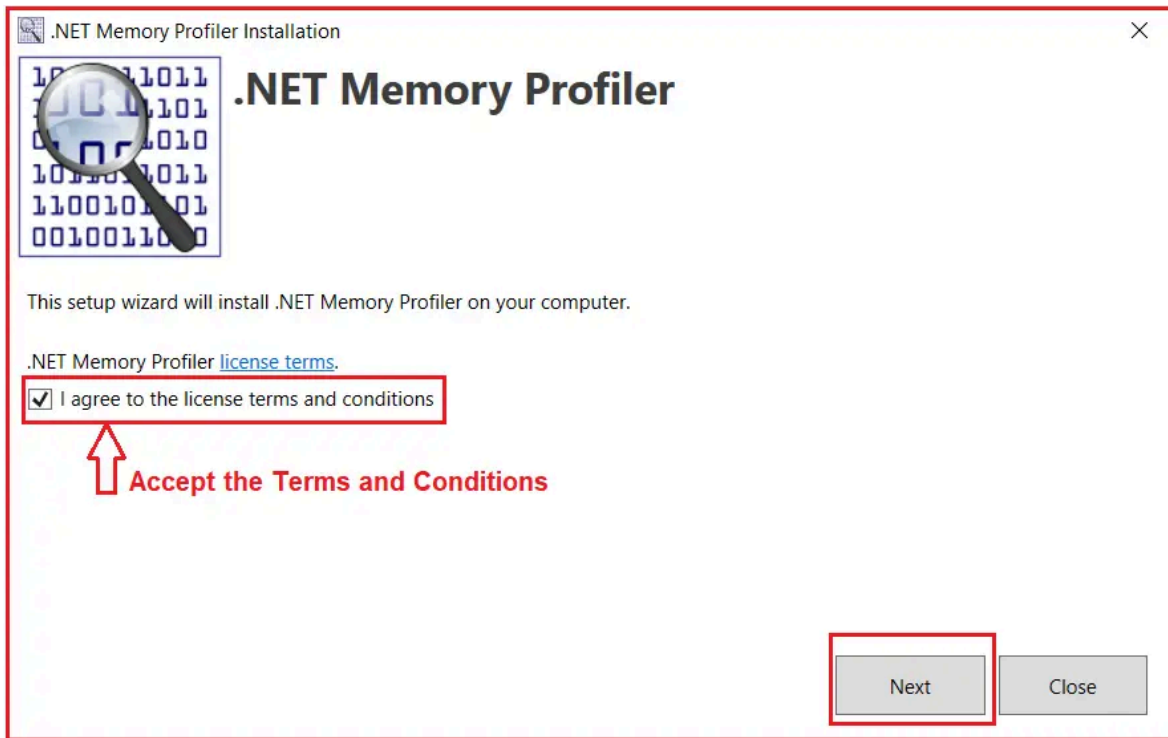
DOWNLOAD

MemProfilerInstaller5_7_26.exe

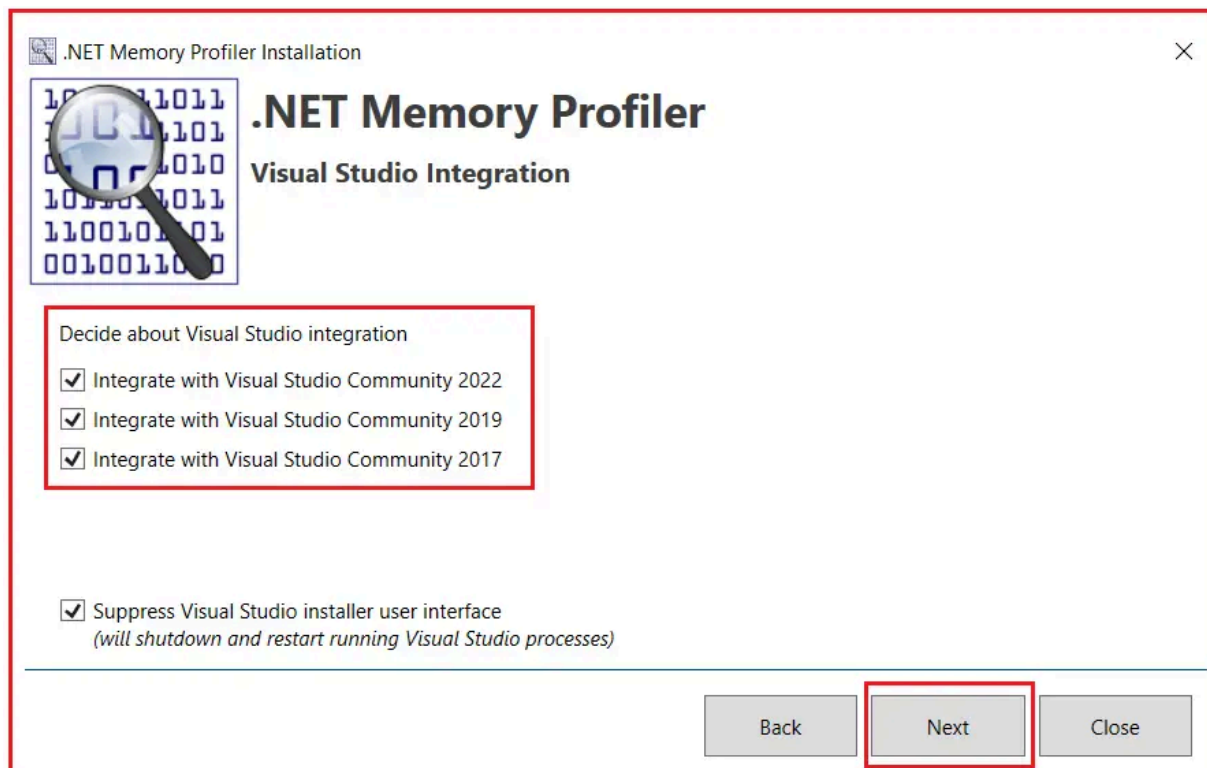
(95,940 KB, version 5.7.26, January 3, 2022)

Click here to Download the EXE

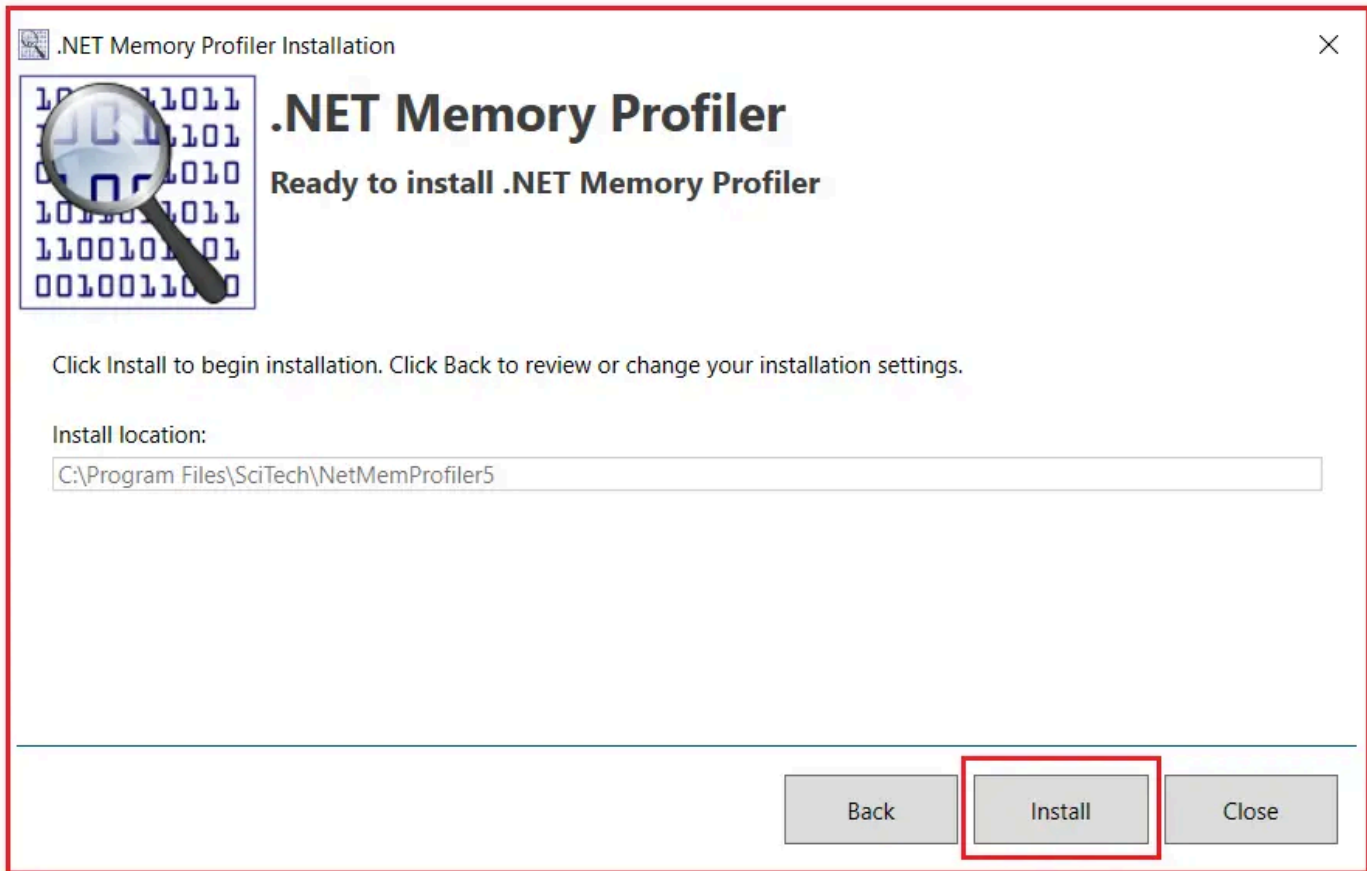
Once you click on the Download button, it will download the .NET Memory Profiler EXE, and once you download the .NET Memory Profiler EXE, click on the downloaded EXE file to install it. Once you click on the EXE file, it will open the following License Agreement Window. Simply check the checkbox and click on the Next button, as shown in the image below.



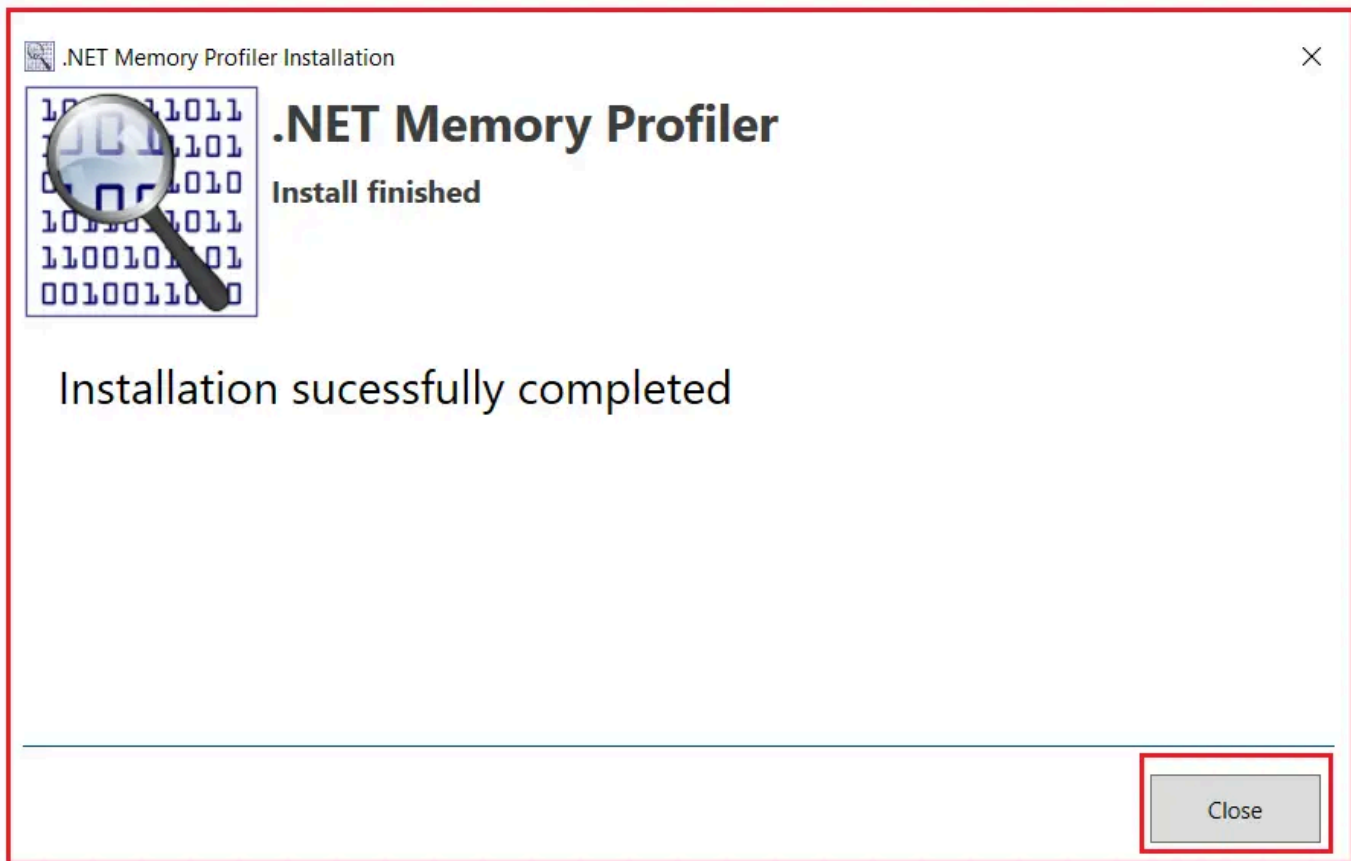
Once you click on the Next button, it will open the following Integrate with Visual Studio window. As I have installed Visual Studio 2017, 2019, and 2022, it shows me all the options, and I want to use this .NET Memory Profiler with all the versions. So, I checked all the checkboxes and then clicked on the Next button, as shown in the image below.



Once you click on the Next button, it will open the Ready to Install window. Click on the Install button as shown in the below image.

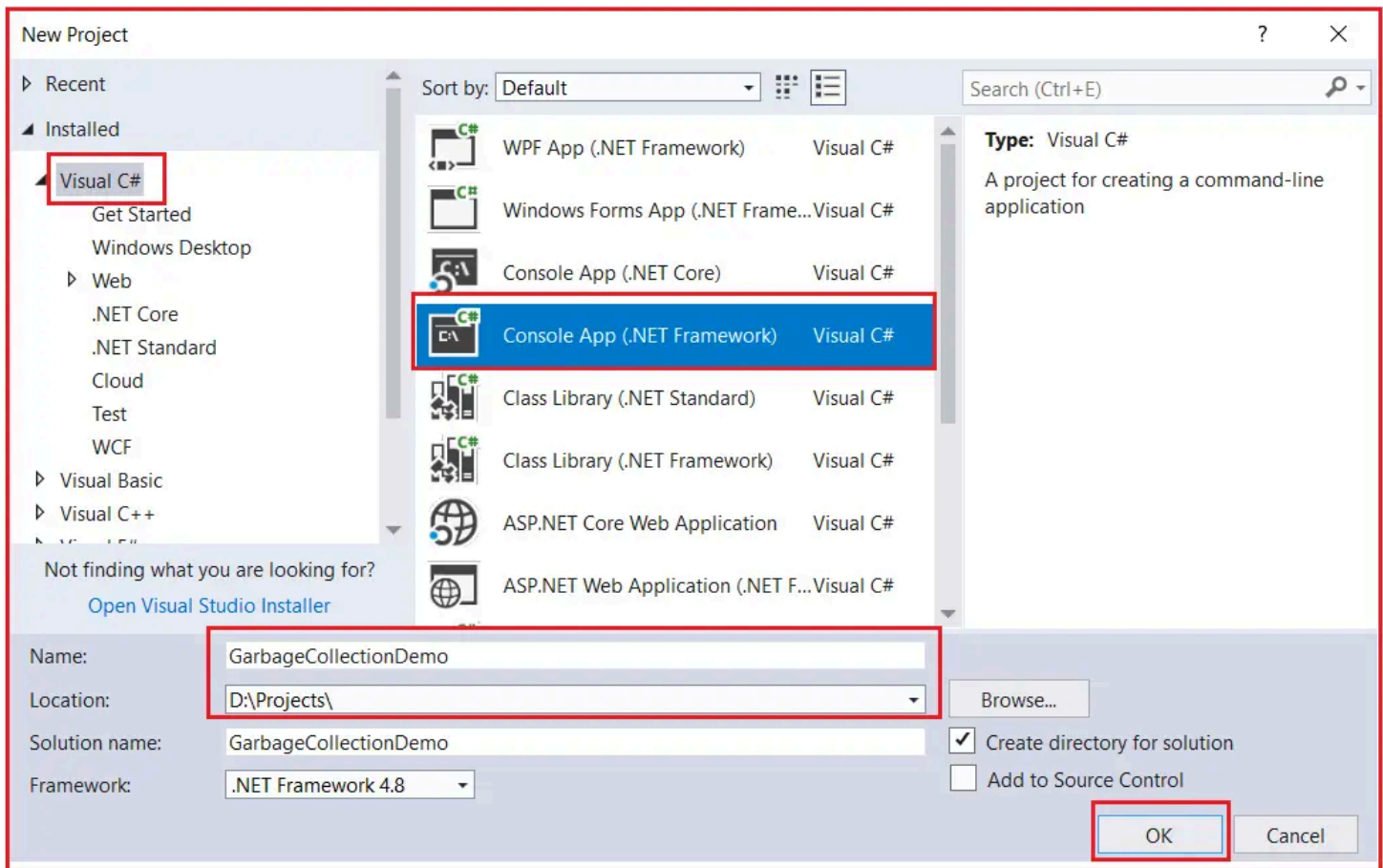


Once you click on the Install button, it will ask you do you want to make changes to this computer, click Yes, so that it will start installing the .NET Memory Profiler on your machine. Once the installation is completed, you will get the following message. Click on the close button to close this.



Creating a C# Console Application:

Now, create a console application with the name **GarbageCollectionDemo** in the **D:\Projects** directory using C# Language, as shown in the image below.



Now, copy and paste the following code into the Program class. Please note that we are not using a destructor here.

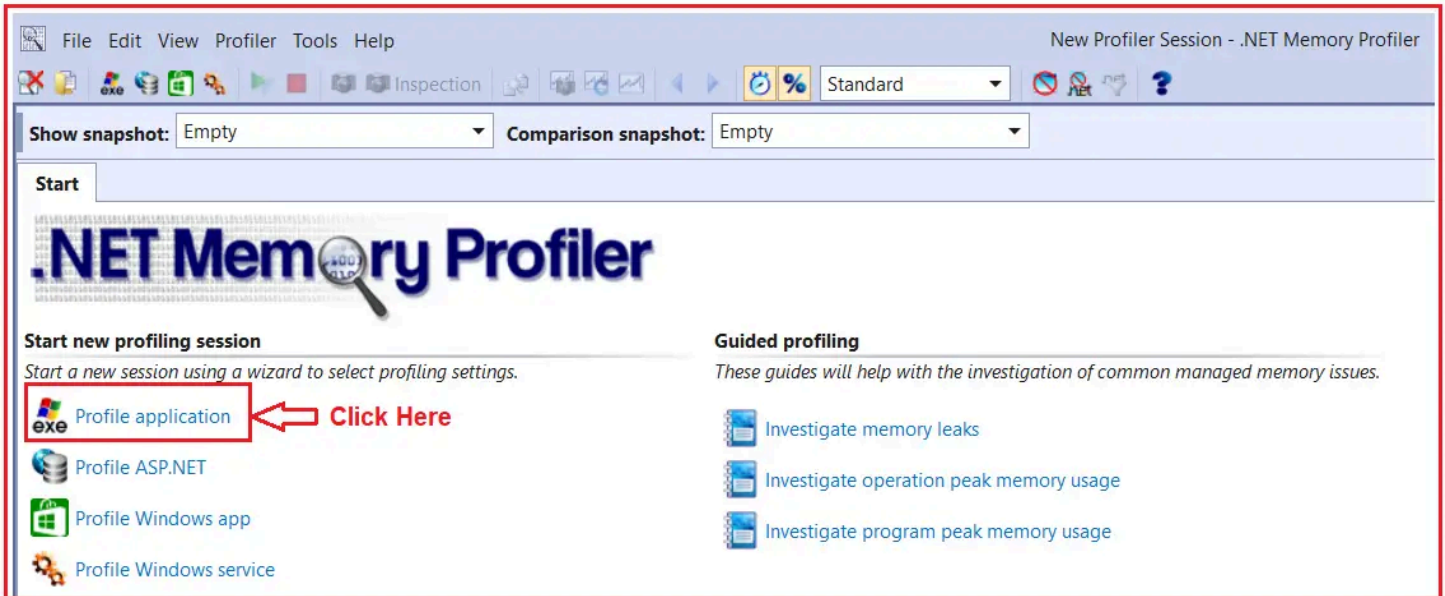
```
using System;
namespace GarbageCollectionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            for (int i = 0; i <= 1000000; i++)
            {
                MyClass1 obj1 = new MyClass1();
                MyClass2 obj2 = new MyClass2();
                MyClass3 obj3 = new MyClass3();
            }
            Console.Read();
        }
    }
    public class MyClass1
    {
    }
    public class MyClass2
    {
    }
    public class MyClass3
    {
    }
}
```

```
}  
{  
}
```

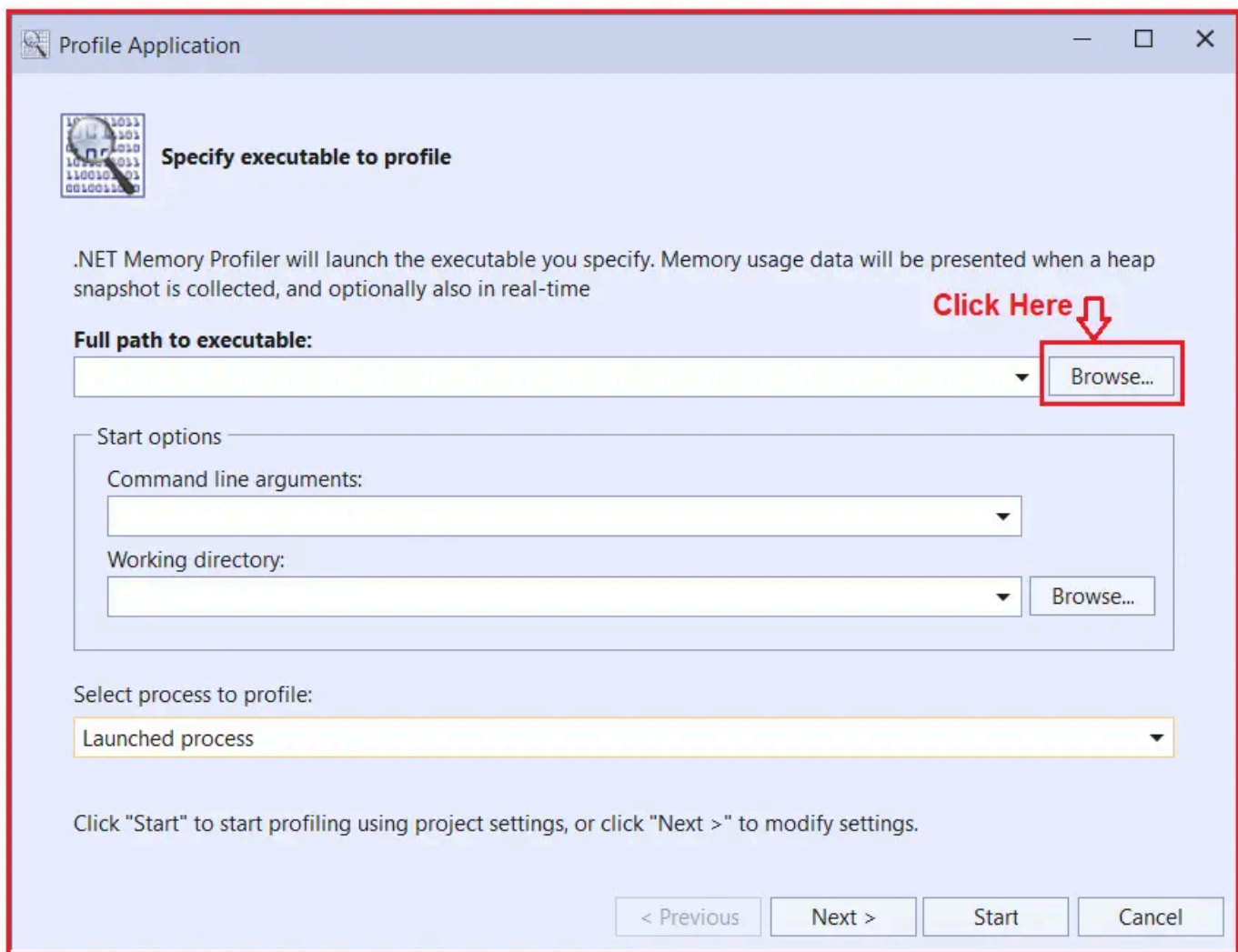
Now, build the solution and make sure there are no errors. Now, we will run this application using the .NET Memory Profiler and see the different generations of garbage collectors.

How do you use the .NET Memory Profiler to run the C# Console Application?

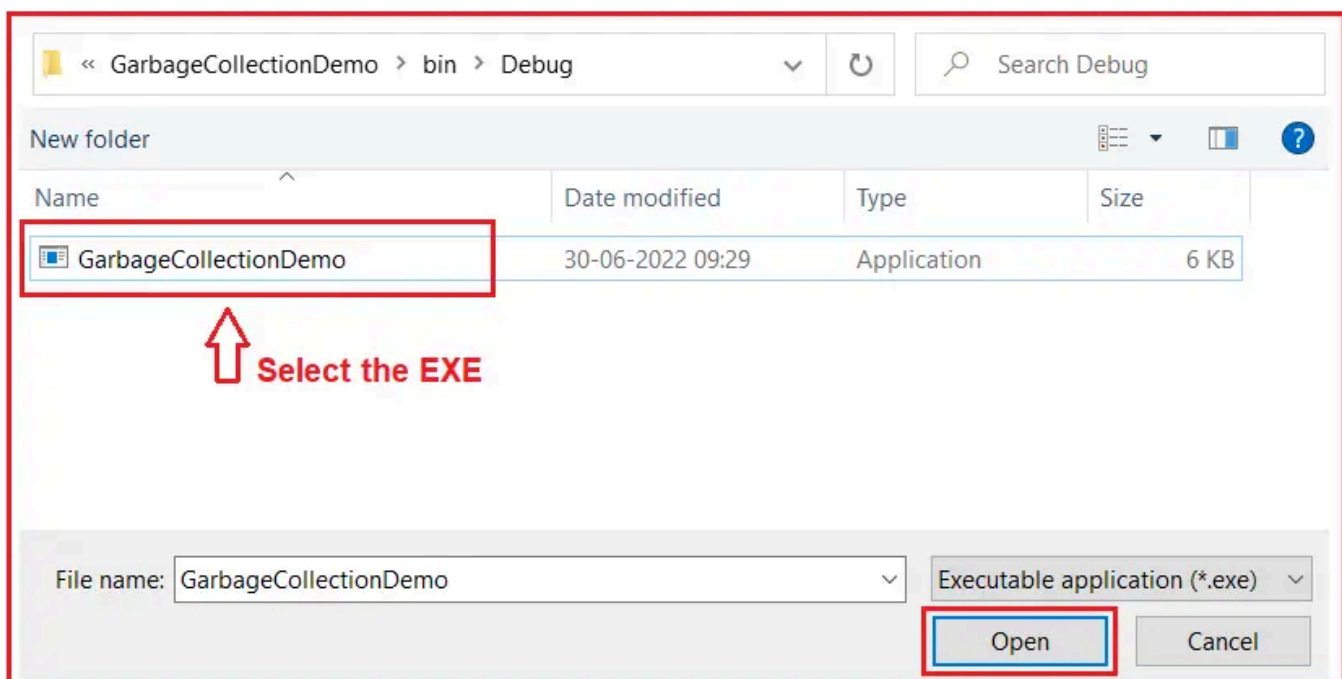
Open the .NET Memory Profiler, and once you open it, you will get the following window. From this window, click on the Profile Application option, as shown in the image below.



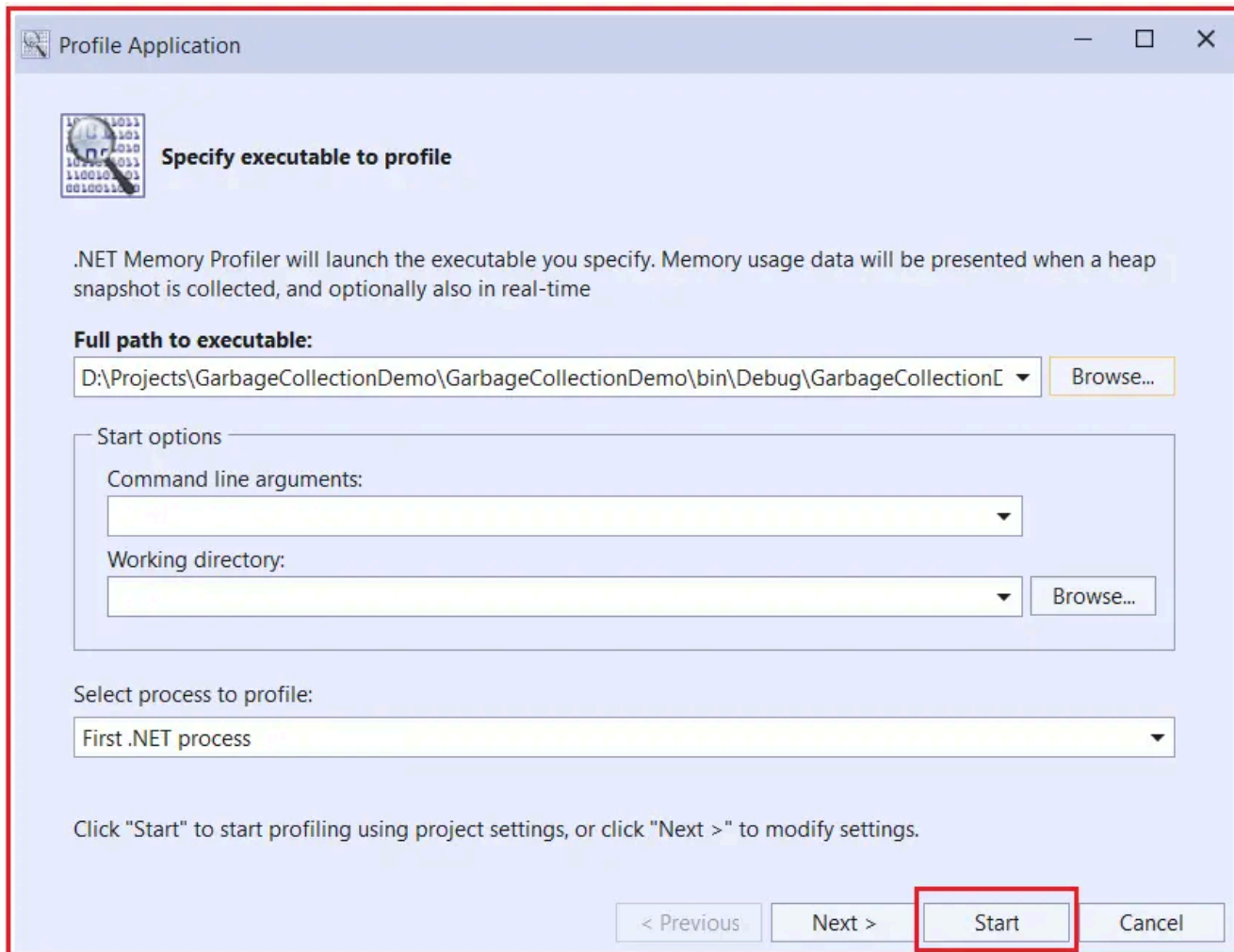
Once you click on the Profile Application Option, it will open the below window. From this window, click on the Browse button, as shown in the below image.



Once you click on the Browse button, select the EXE, i.e., present inside the Bin=>Deubg folder or your project, and click on the Open Folder as shown in the below image.



Once you click on the Open button, it will take you back to the Profile Application window, and here, you need to click on the Start button, as shown in the below image.



The screenshot shows a window titled "Profile Application" with a standard Windows title bar (minimize, maximize, close buttons). Inside the window, there is a section titled "Specify executable to profile" with a small icon of a magnifying glass over a document. Below this title, a text box explains: ".NET Memory Profiler will launch the executable you specify. Memory usage data will be presented when a heap snapshot is collected, and optionally also in real-time".

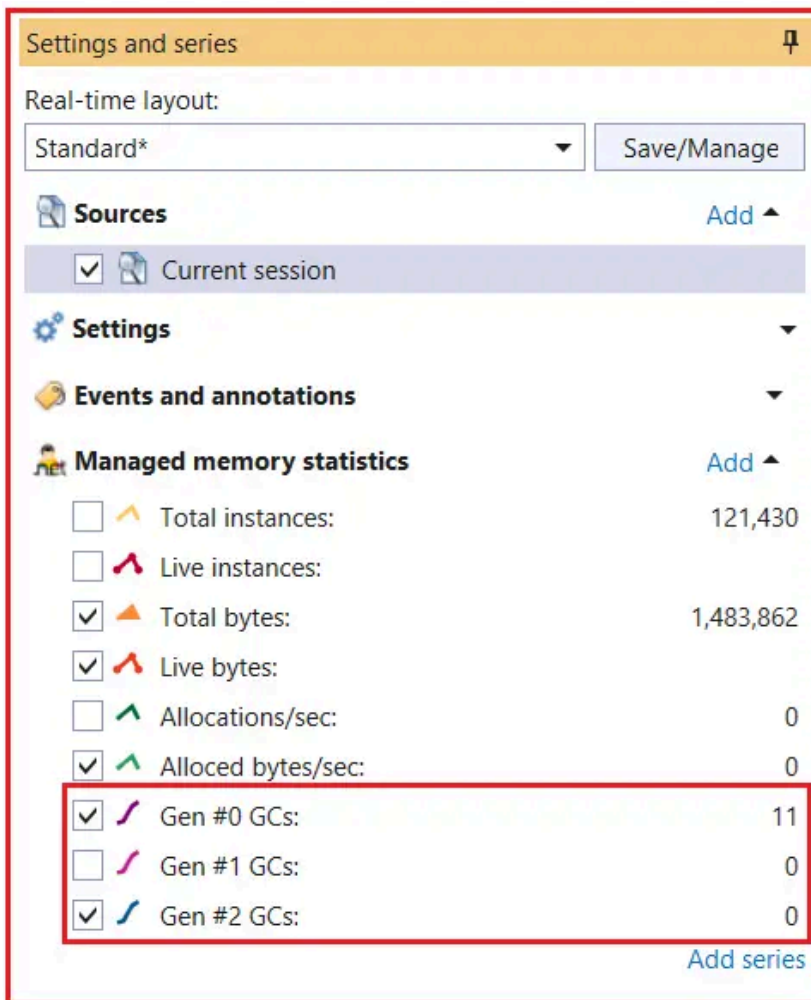
Under the heading "Full path to executable:", there is a text input field containing the path "D:\Projects\GarbageCollectionDemo\GarbageCollectionDemo\bin\Debug\GarbageCollectionE" and a "Browse..." button to its right.

Below this, there is a "Start options" section enclosed in a rounded rectangle. It contains two sub-sections: "Command line arguments:" with a text input field, and "Working directory:" with a text input field and a "Browse..." button to its right.

Further down, there is a "Select process to profile:" section with a dropdown menu currently showing "First .NET process".

At the bottom of the window, there is a line of text: "Click 'Start' to start profiling using project settings, or click 'Next >' to modify settings." Below this text are four buttons: "< Previous", "Next >", "Start", and "Cancel". The "Start" button is highlighted with a red rectangular border.

Once you click on the Start button, it will start executing your console application, and you can also observe the generations. You can see most of the objects are in generation 0.



So, the more objects in generation 0, the better the performance and the more the memory will be utilized in an optimal manner.

How does using a Destructor in a Class end up in a Double Garbage Collector Loop?

As we already discussed, garbage collectors will only clean up the managed code. In other words, for any unmanaged code, for those codes to be cleaned up, it has to be provided by unmanaged code, and the garbage collector does not have any control over them to clean up the memory.

For example, let's say you have a MyClass class in VB6. Then you have to expose some function, let's say CleanUp(), and in that function, you have to write the logic to clean up the unmanaged code. You need to call that method (CleanUp()) from your dot net code to initiate the clean-up.

The location from where you would like to call the Clean-Up is the destructor of a class. This looks to be the best place to write the clean-up code. But, a big problem is associated with it when you write clean-up in a destructor. Let us understand what the problem is.

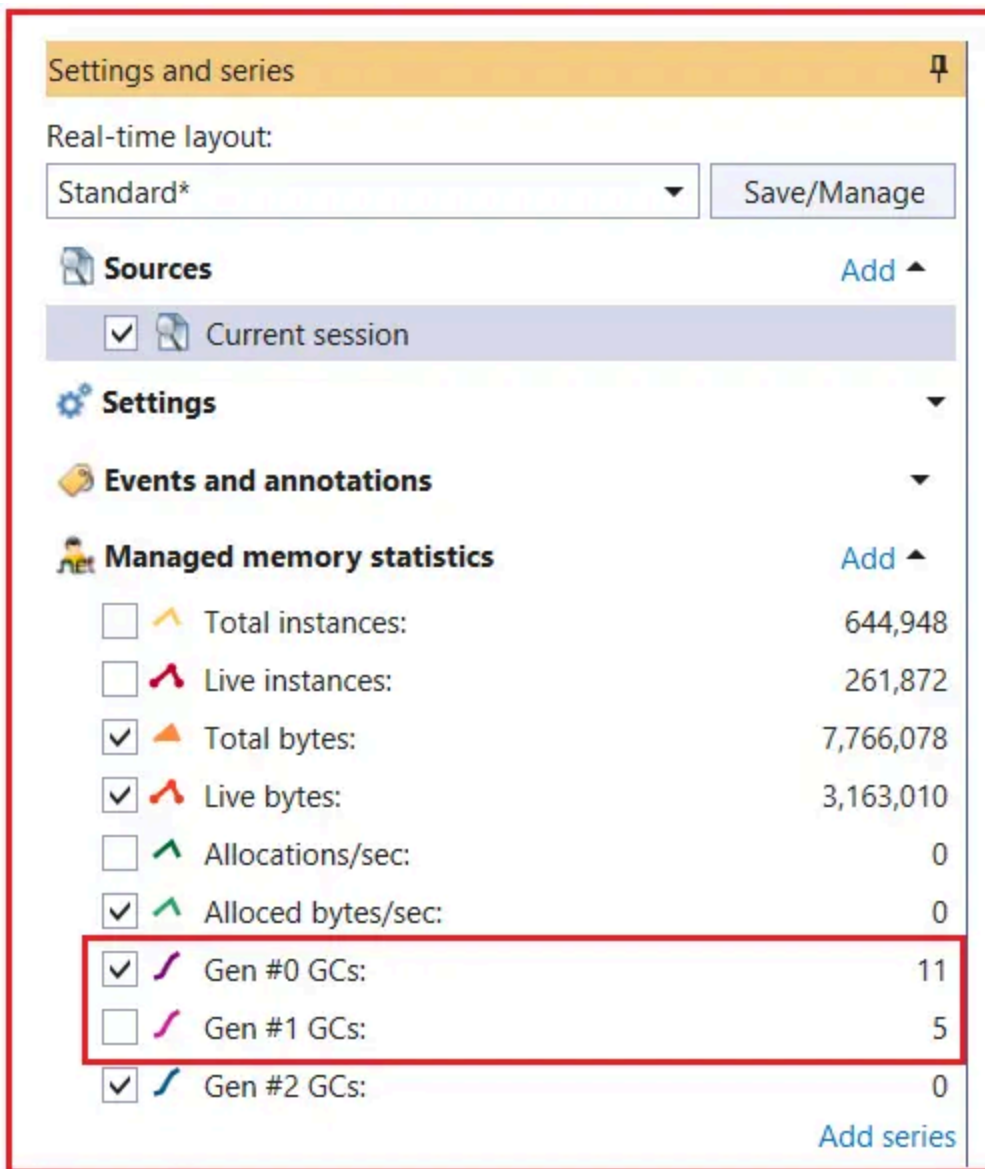
When you define a destructor in your class, the Garbage Collector before destroying the object, will go and ask the question to class, do you have a destructor, if you have a destructor, then move the object to the next generation bucket. In other words, it will not clean up the object having a destructor at that moment itself, even though it is not used. So, it will wait for the destructor to run, and then it will go and clean up the object. Because of this, you will find more objects in Generation 1 and Generation 2 than in Generation 0.

Example using Destructor to Destroy the Unmanaged Resources:

Please have a look at the below code. This is the same example as the previous one, except we have added the respective destructors in the class.

```
using System;
namespace GarbageCollectionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            for (int i = 0; i <= 1000000; i++)
            {
                MyClass1 obj1 = new MyClass1();
                MyClass2 obj2 = new MyClass2();
                MyClass3 obj3 = new MyClass3();
            }
            Console.Read();
        }
    }
    public class MyClass1
    {
        ~MyClass1()
        {
            //Here, you need to write the code for
            //Unmanaged resource clean up
        }
    }
    public class MyClass2
    {
        ~MyClass2()
        {
            //Here, you need to write the code for
            //Unmanaged resource clean up
        }
    }
    public class MyClass3
    {
        ~MyClass3()
        {
            //Here, you need to write the code for
            //Unmanaged resource clean up
        }
    }
}
```

Now, rebuild the solution. Now, close the .NET Memory Profile and follow the same steps to run the console application using this .NET Memory Profiler. This time, you will observe that some of the objects are in generation 1 also, as shown in the below image.



So, if you are writing the clean-up code in your destructor, you will create objects in Generation 1 and Generation 2, which means you are not utilizing the memory properly.

How to Overcome the above Problem?

This problem can be overcome by using something called the Finalized Dispose pattern. To implement this, your class should implement the `IDisposable` interface and provide the implementation for the `Dispose` method. Within the `Dispose` method, you need to write the clean-up code for unmanaged objects, and in the end, you need to call `GC.SuppressFinalize(true)` method by passing `true` as the input value. This method suppresses any kind of destructor and just goes and cleans up the objects. For a better understanding, please have a look at the following image.

```

public class MyClass1 : IDisposable
{
    #region IDisposable Support
    private bool disposedValue = false; // To detect redundant calls
    protected virtual void Dispose(bool disposing)
    {
        if (!disposedValue)
        {
            if (disposing)
            {
                // TODO: dispose managed state (managed objects).
            }

            // TODO: free unmanaged resources (unmanaged objects) and override a finalizer below.
            // TODO: set large fields to null.

            disposedValue = true;
        }
    }

    // TODO: override a finalizer only if Dispose(bool disposing) above has code to free unmanaged resources.
    // ~MyClass1() {
    //     // Do not change this code. Put cleanup code in Dispose(bool disposing) above.
    //     Dispose(false);
    // }

    // This code added to correctly implement the disposable pattern.
    public void Dispose()
    {
        // Do not change this code. Put cleanup code in Dispose(bool disposing) above.
        Dispose(true);
        // TODO: uncomment the following line if the finalizer is overridden above.
        // GC.SuppressFinalize(this);
    }
    #endregion
}

```

Once you have used the object, then you need to call the Dispose method so that the double garbage collector loop will not happen, as shown below.

```

MyClass1 obj1 = new MyClass1();
obj1.Dispose();

```

Example using Dispose Pattern to Destroy the Unmanaged Object in C#:

```

using System;
namespace GarbageCollectionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            for (int i = 0; i <= 1000000; i++)
            {

```

```

        MyClass1 obj1 = new MyClass1();
        obj1.Dispose();
        MyClass2 obj2 = new MyClass2();
        obj2.Dispose();
        MyClass3 obj3 = new MyClass3();
        obj3.Dispose();
    }
    Console.Read();
}
}
public class MyClass1 : IDisposable
{
    #region IDisposable Support
    private bool disposedValue = false; // To detect redundant calls
    protected virtual void Dispose(bool disposing)
    {
        if (!disposedValue)
        {
            if (disposing)
            {
                // TODO: dispose managed state (managed objects).
            }
            // TODO: free unmanaged resources (unmanaged objects) and over
finalizer below.
            // TODO: set large fields to null.
            disposedValue = true;
        }
    }
    // TODO: override a finalizer only if Dispose(bool disposing) above ha
free unmanaged resources.
    ~MyClass1()
    {
        // Do not change this code. Put cleanup code in Dispose(bool disp
above.
        Dispose(false);
    }
    // This code added to correctly implement the disposable pattern.
    public void Dispose()
    {
        // Do not change this code. Put cleanup code in Dispose(bool disp
above.
        Dispose(true);
        // TODO: uncomment the following line if the finalizer is override
GC.SuppressFinalize(this);
    }
    #endregion
}
public class MyClass2 : IDisposable
{
    #region IDisposable Support
    private bool disposedValue = false; // To detect redundant calls

```

```

protected virtual void Dispose(bool disposing)
{
    if (!disposedValue)
    {
        if (disposing)
        {
        }
        disposedValue = true;
    }
}
// TODO: override a finalizer only if Dispose(bool disposing) above has
free unmanaged resources.
~MyClass2()
{
    // Do not change this code. Put cleanup code in Dispose(bool disposing)
above.
    Dispose(false);
}
// This code added to correctly implement the disposable pattern.
public void Dispose()
{
    // Do not change this code. Put cleanup code in Dispose(bool disposing)
above.
    Dispose(true);
    // TODO: uncomment the following line if the finalizer is overridden.
    GC.SuppressFinalize(this);
}
#endregion
}
public class MyClass3 : IDisposable
{
    #region IDisposable Support
    private bool disposedValue = false;
    protected virtual void Dispose(bool disposing)
    {
        if (!disposedValue)
        {
            if (disposing)
            {
            }

            disposedValue = true;
        }
    }

    ~MyClass3()
    {
        Dispose(false);
    }
    public void Dispose()
    {

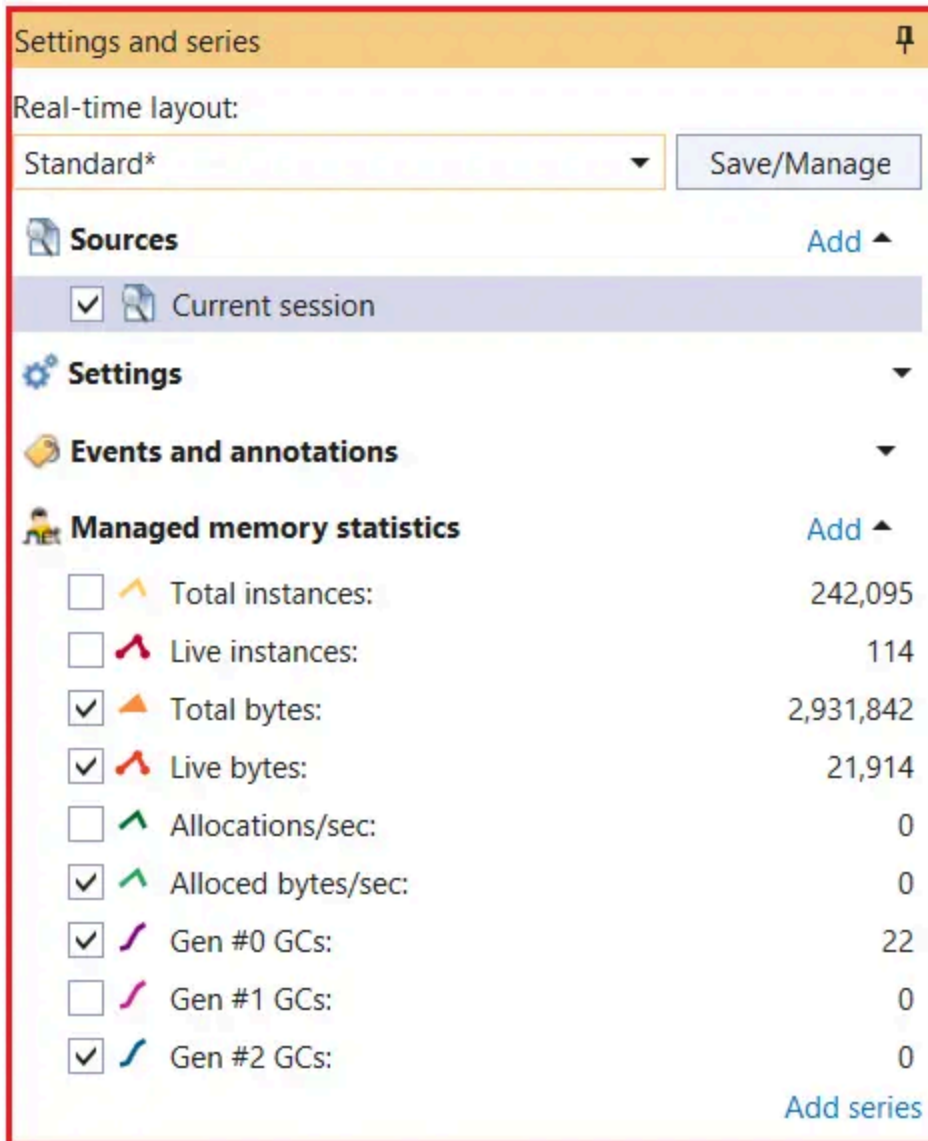
```

```

        Dispose(true);
        GC.SuppressFinalize(this);
    }
    #endregion
}
}

```

Now, rebuild the solution. Close the .NET Memory Profile and follow the same steps to run the console application using this .NET Memory Profiler. This time, you will observe that the objects are created in generation 0 only, which improves the performance of your application by utilizing the memory effectively.



Now, the question that should come to your mind is why the destructor is there. The reason is that, as a developer, you may forget to call the Dispose method once you use the object. In that case, the destructor will invoke it, and it will go and clean up the object.

In this article, I will explain the **Differences Between Finalize and Dispose in C#** with Example. In this article, I try to explain Garbage Collection in the .NET Framework with examples. I hope you enjoy this Garbage Collection in .NET Framework article, and I also hope that now you understand how the garbage collector works in C#.